

Отчет по лабораторной работе №13

**Программирование в командном процессоре ОС UNIX. Ветвления и
циклы**

Виноградова Мария Андреевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Выводы	14

Список иллюстраций

2.1	Пишем командный файл	6
2.2	Смотрим реализацию	7
2.3	Пишем код	8
2.4	Пишем командный файл	9
2.5	Смотрим реализацию	10
2.6	Пишем командный файл	11
2.7	Смотрим реализацию	11
2.8	Пишем командный файл	12
2.9	Смотрим реализацию	12
2.10	Проверяем архив	13

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Выполнение лабораторной работы

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с некоторыми ключами, а затем ищет в указанном файле нужные строки, определяемые ключом `-p` (рис. 2.1).

```
1 #!/bin/bash
2
3 while getopts ":i:o:p:cl" opt; do
4     case ${opt} in
5         i)|
6             inputfile=$OPTARG
7             ;;
8         o)
9             outputfile=$OPTARG
10            ;;
11        p)
12            pattern=$OPTARG
13            ;;
14        c)
15            case_sensitive=true
16            ;;
17        l)
18            line_numbers=true
19            ;;
20        \?)
21            echo "Неверный параметр: $OPTARG" 1>&2
22            exit 1
23            ;;
24        :)
25            echo "Отсутствует значение для параметра: $OPTARG"
26            1>&2
27            exit 1
28            ..
```

Рис. 2.1: Пишем командный файл

Проверяем работу командного файла: (рис. 2.2).

```

[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text
любые случайные или нет
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -n
Неверный параметр: n
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -n
Неверный параметр: n
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -n
Неверный параметр: n
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -n
Неверный параметр: n
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -l
2:любые случайные или нет
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -C
Неверный параметр: C
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -c
Неверный параметр:
[mavinogradova@vbox lab13]$
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -c
Неверный параметр:
[mavinogradova@vbox lab13]$
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -C
Неверный параметр:
[mavinogradova@vbox lab13]$
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -C
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -C
[mavinogradova@vbox lab13]$
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -c
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -l -c
2:любые случайные или нет
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -l -c
Неверный параметр: c
[mavinogradova@vbox lab13]$ ./1.sh -p "нет" -i text -l -c
2:любые случайные или нет
[mavinogradova@vbox lab13]$ ./1.sh -p "Нет" -i text -l -c
2:любые случайные или нет

```

Рис. 2.2: Смотрим реализацию

2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю (рис. 2.3).

Рис. 2.3: Пишем код

2.1. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено (рис. 2.4).

```
report.md x 1.sh x text x 2
1 #!/bin/bash
2
3 ./2
4
5 case $? in
6     0)
7         echo "Число равно нулю"
8         ;;
9     1)
10        echo "Число больше нуля"
11        ;;
12    2)
13        echo "Число меньше нуля"
14        ;;
15 esac
```

Рис. 2.4: Пишем командный файл

Проверяем работу командного файла и самого кода: (рис. 2.5).

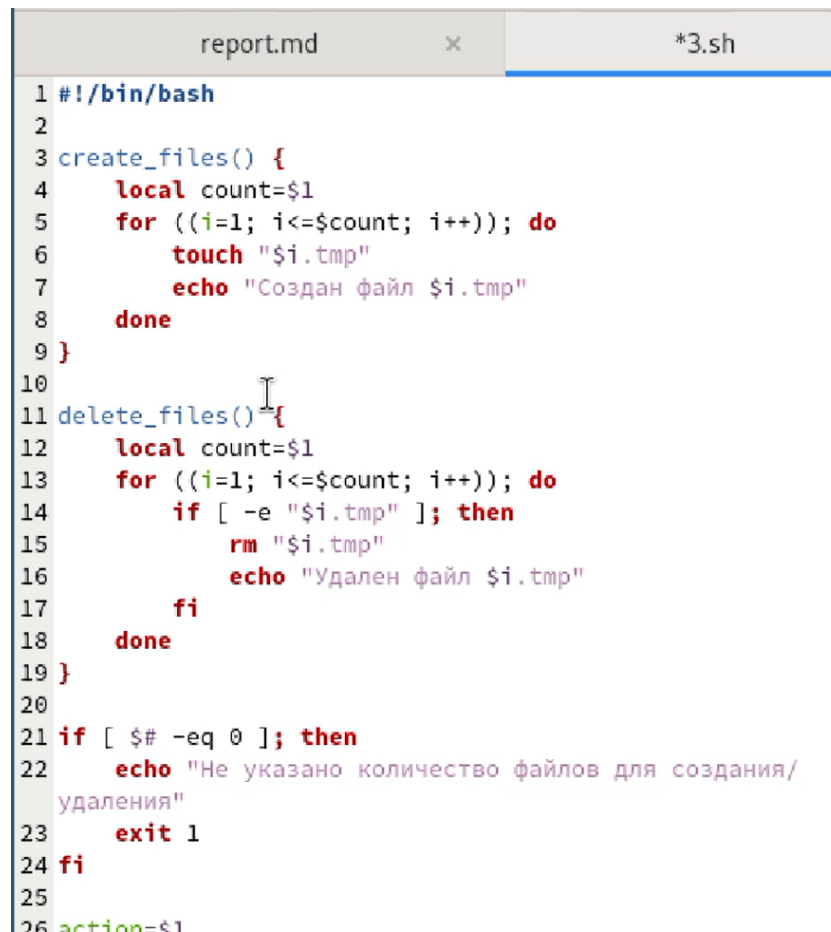
```

[mavinogradova@vbox lab13]$ gcc 2.c -o 2
[mavinogradova@vbox lab13]$ ./2
Введите число: 5
Число больше нуля
[mavinogradova@vbox lab13]$ ./2.sh
./2.sh: строка 3: ./2: Нет такого файла или каталога
[mavinogradova@vbox lab13]$ ./2.sh
Введите число: 7
Число больше нуля
Число больше нуля
[mavinogradova@vbox lab13]$ ./2.sh
Введите число: -5
Число меньше нуля
Число меньше нуля
[mavinogradova@vbox lab13]$ ./2.sh
Введите число: 0
Число равно нулю
Число равно нулю
[mavinogradova@vbox lab13]$

```

Рис. 2.5: Смотрим реализацию

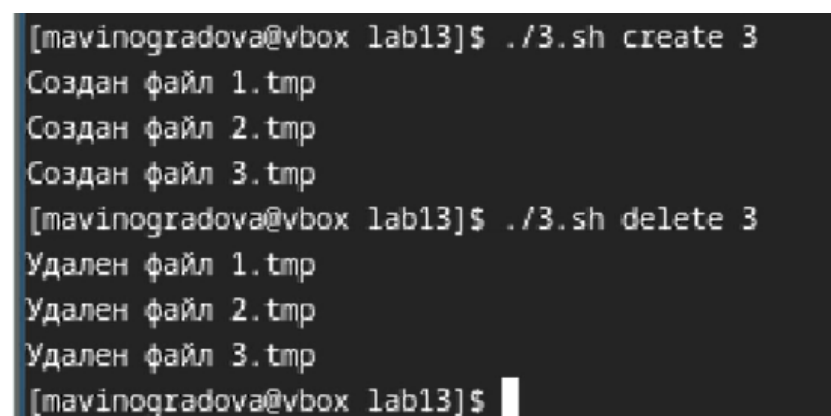
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n . Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют). (рис. 2.6).



```
1 #!/bin/bash
2
3 create_files() {
4     local count=$1
5     for ((i=1; i<=$count; i++)); do
6         touch "$i.tmp"
7         echo "Создан файл $i.tmp"
8     done
9 }
10
11 delete_files() {
12     local count=$1
13     for ((i=1; i<=$count; i++)); do
14         if [ -e "$i.tmp" ]; then
15             rm "$i.tmp"
16             echo "Удален файл $i.tmp"
17         fi
18     done
19 }
20
21 if [ $# -eq 0 ]; then
22     echo "Не указано количество файлов для создания/
    удаления"
23     exit 1
24 fi
25
26 action=$1
```

Рис. 2.6: Пишем командный файл

Проверяем работу командного файла: (рис. 2.7).

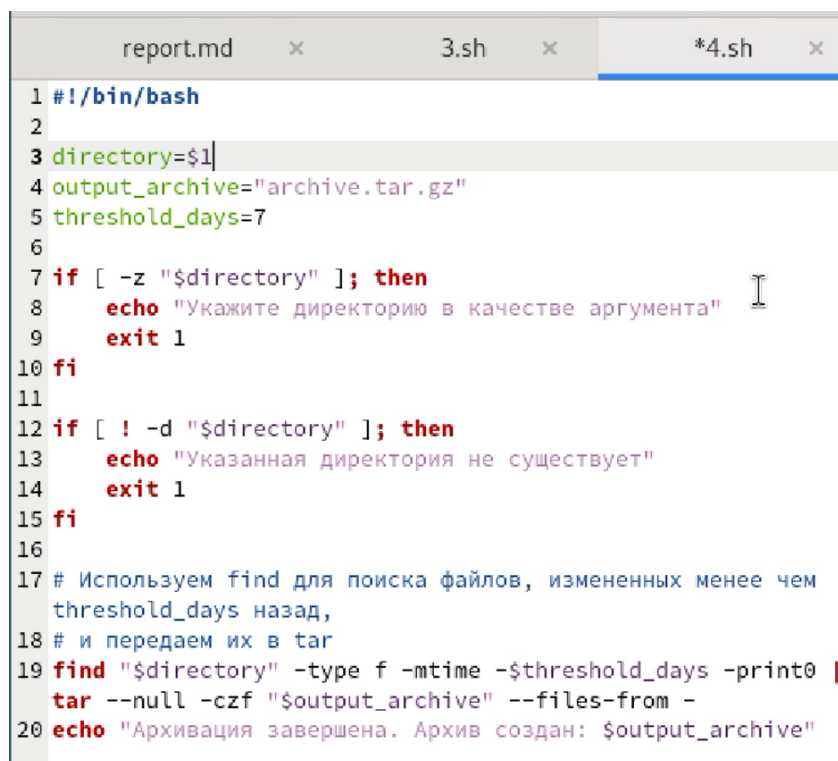


```
[mavinogradova@vbox lab13]$ ./3.sh create 3
Создан файл 1.tmp
Создан файл 2.tmp
Создан файл 3.tmp
[mavinogradova@vbox lab13]$ ./3.sh delete 3
Удален файл 1.tmp
Удален файл 2.tmp
Удален файл 3.tmp
[mavinogradova@vbox lab13]$
```

Рис. 2.7: Смотрим реализацию

4. Написать командный файл, который с помощью команды tar запаковывает

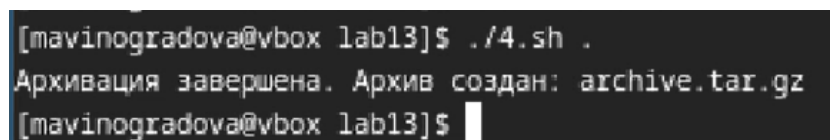
в архив все файлы в указанной директории. (рис. 2.8).



```
1 #!/bin/bash
2
3 directory=$1
4 output_archive="archive.tar.gz"
5 threshold_days=7
6
7 if [ -z "$directory" ]; then
8     echo "Укажите директорию в качестве аргумента"
9     exit 1
10 fi
11
12 if [ ! -d "$directory" ]; then
13     echo "Указанная директория не существует"
14     exit 1
15 fi
16
17 # Используем find для поиска файлов, измененных менее чем
18 #   threshold_days назад,
19 #   и передаем их в tar
20 find "$directory" -type f -mtime -$threshold_days -print0 |
21 tar --null -czf "$output_archive" --files-from -
22 echo "Архивация завершена. Архив создан: $output_archive"
```

Рис. 2.8: Пишем командный файл

Проверяем работу командного файла: (рис. 2.9).



```
[mavinogradova@vbox lab13]$ ./4.sh .
Архивация завершена. Архив создан: archive.tar.gz
[mavinogradova@vbox lab13]$
```

Рис. 2.9: Смотрим реализацию

Проверяем файлы в директории (распаковываем архив): (рис. 2.10).

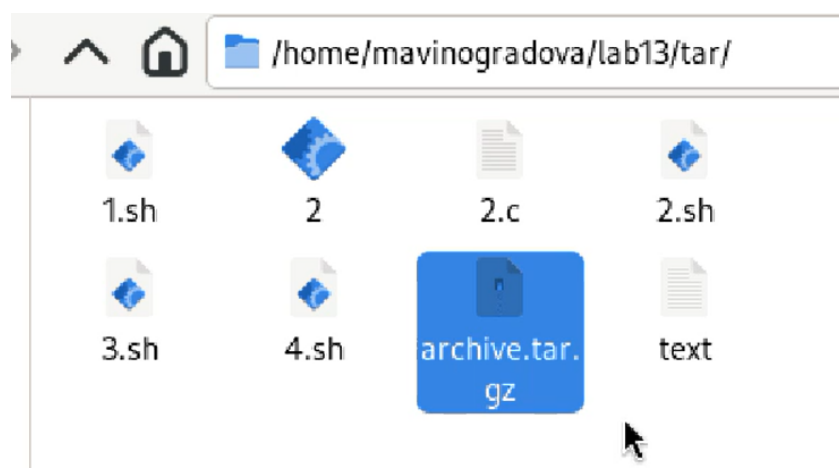


Рис. 2.10: Проверяем архив

3 Выводы

В ходе лабораторной работы изучены основы программирования в оболочке UNIX, освоено создание сложных командных файлов с использованием условных операторов, циклов и управляющих конструкций. Приобретённые навыки позволяют автоматизировать задачи в UNIX-подобных системах, повышая эффективность работы с командной строкой.